20-单片机 2

大家晚上好，上节课，我们讲了新建工程和启动文件的开头。
前面我们提到了 SR 寄存器

3.7 CPU 控制寄存器

寄存器 3-1： SR：CPU 状态寄存器

R/W-0	R/W-0	R/W-0	R/W-0	R/C-0	R/C-0	R-0	R/W-0
OA(1)	OB(1)	SA(1,4)	SB(1,4)	OAB(1)	SAB(1)	DA(1)	DC
bit 15							bit 8

R/W-0(2,3)	R/W-0(2,3)	R/W-0(2,3)	R-0	R/W-0	R/W-0	R/W-0	R/W-0
	IPL<2:0>		RA	N	OV	Z	C
bit 7							bit 0

图注：	C = 可清零位		
R = 可读位	W = 可写位	U = 未实现位，读为 0	
-n = POR 时的值	1 = 置 1	0 = 清零	x = 未知

这个 SR 寄存器需要重点了解一下，这里的 IPL 是和单片机中断优先有关。大家需要自己去了解这个单片机的优先级。

```
mov SR,W0
mov #0x00e0,W1
ior W0,W1,W0
mov W0,SR
```

这几句话的作用就是将 IPL<2:0>设置为 111，禁止用户中断(关中断)。当用户实际中断优先级设置大于这个 IPL，当前中断才能响应，反之这个中断不会响应。所以当 IPL 设置最大时，就相当于关闭了所有中断。IPL=7 意思就是关闭优先级从 0-7 的所有用户中断。

寄存器相关位置 1， 我们通常使用 ior 或操作；寄存器相关位清 0， 我们通常使用 and 与操作。

```
mov     SR,W0
mov     #0x00e0,W1
ior     W0,W1,W0
mov     W0,SR
```

现在来问大家，如果 IPL<2:0>=0,这段代码如何写？会写的可以写一下

```
mov     sr,w0
mov     #0xFF1F,w1
and     w0,w1,w0
mov     w0,sr
```

对的，这是单片机软件中常用的对寄存器赋值方法，不论是汇编还是 C 语言，都是这个思路。

我们继续向下啊

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
__reset:
    mov     SR, W0
    mov     #0x00e0, W1
    ior     W0, W1, W0
    mov     W0, SR

    mov     RCON, W0
    mov     #0xffdf, W1
    and     W0, W1, W0
    mov     W0, RCON
```

现在大家再来看红框部分，说说看是什么意思？
RCON 寄存器第 5bit 清零

问：Rcon 是啥？

寄存器 6-1：　RCON：复位控制寄存器 (1)

R/W-0	R/W-0	U-0	U-0	R/W-0	U-0	R/W-0	R/W-0
TRAPR	IOPUWR	—	—	VREGSF	—	CM	VREGS
bit 15							bit 8

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-1	R/W-1
EXTR	SWR	SWDTEN(2)	WDTO	SLEEP	IDLE	BOR	POR
bit 7							bit 0

bit 13-12　**未实现：** 读为 0
bit 11　**VREGSF：** 休眠模式下闪存稳压器待机位
1 = 在休眠模式下闪存稳压器继续工作
0 = 在休眠模式下闪存稳压器进入待机模式
bit 10　**未实现：** 读为 0
bit 9　**CM：** 配置不匹配标志位
1 = 发生了配置不匹配复位
0 = 未发生配置不匹配复位
bit 8　**VREGS：** 休眠模式下稳压器待机位
1 = 在休眠模式下稳压器继续工作
0 = 在休眠模式下稳压器进入待机模式
bit 7　**EXTR：** 外部复位（$\overline{MCLR}$）引脚位
1 = 发生了主复位（引脚）复位
0 = 未发生主复位（引脚）复位
bit 6　**SWR：** 软件 RESET（指令）标志位
1 = 执行了 RESET 指令
0 = 未执行 RESET 指令
bit 5　**SWDTEN：** 软件使能 / 禁止 WDT 位 (2)
1 = 使能 WDT
0 = 禁止 WDT
bit 4　**WDTO：** 看门狗定时器超时标志位
1 = 发生了 WDT 超时
0 = 未发生 WDT 超时

```
    mov     RCON, W0
    mov     #0xffdf, W1
    and     W0, W1, W0
    mov     W0, RCON
```

这段话的作用就是关闭看门狗功能。

问：使能看门狗？
答：清 0 是关闭看门狗功能。

是不是感觉汇编似乎没有那么难。启动文件中的汇编不多，了解一下的份量刚刚好。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
__reset:
    mov     SR,W0
    mov     #0x00e0,W1
    ior     W0,W1,W0
    mov     W0,SR

    mov     RCON,W0
    mov     #0xffdf,W1
    and     W0,W1,W0
    mov     W0,RCON

    mov     #4,W1
    clr     W0
    dec     W0,W0
    bra     NZ,$-2
    dec     W1,W1
    bra     NZ,$-8
```

接下来红框中是一段循环语句，总共呢，有六条指令。当然了，这里涉及到了 5 条新的指令。不要畏惧，我们逐条来分析一下。

第一条指令，现在应该很清楚了，就是把立即数 4 存到 W1 寄存器中

第二条指令，clr　W0　　clr 其实就是 clear 的简写，也就是清除的意思。所以，这条指令的意思呢，就是把 W0 寄存器清 0。

表 3-4:　　逻辑指令

汇编语法		说明	字数	周期数	页码
AND	f {,WREG}(1)	目标寄存器 = f .AND.WREG	1	1(2)	112
AND	#lit10,Wn	Wn = lit10 .AND.Wn	1	1	113
AND	Wb,#lit5,Wd	Wd = Wb .AND. lit5	1	1	114
AND	Wb,Ws,Wd	Wd = Wb .AND.Ws	1	1(2)	115
CLR	f	f = 0x0000	1	1	184
CLR	WREG	WREG = 0x0000	1	1	184
CLR	Wd	Wd = 0x0000	1	1	185
COM	f {,WREG}(1)	目标寄存器 = $\bar{f}$	1	1(2)	189
COM	Ws,Wd	Wd = $\overline{Ws}$	1	1(2)	190

其实呢，我们查一下指令表。也可以知道指令的意思。也就是说，指令不需要去背，能够查指令表就行。

```
mov     #4,W1
clr     W0
dec     W0,W0
bra     NZ,$-2
dec     W1,W1
bra     NZ,$-8
```

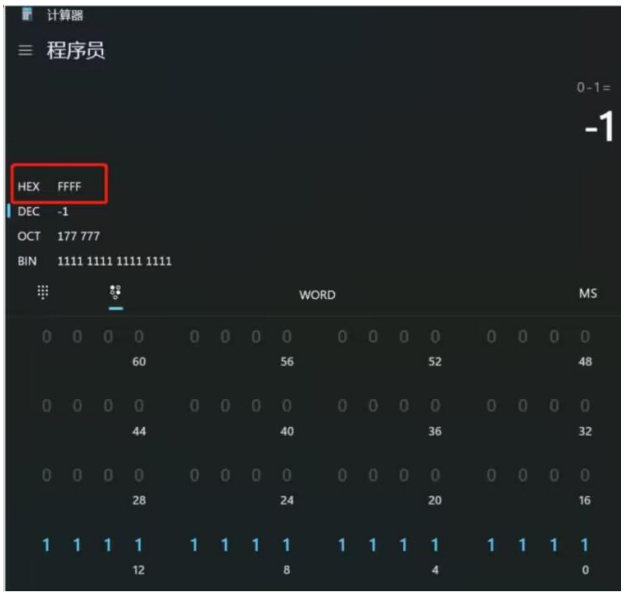
所以，第二条指令就是把 W0 寄存器的内容进行清 0。

接着，我们来看第三条语句。dec　W0,W0　dec 是减的缩写。这里呢，也就是一条减法指令。这条指令的意思呢，是要把 W0 的内容减 1。

DEC	Ws,Wd	Wd = Ws – 1	1	1(5)	218

大家看，原始 W0 中是 0，现在对它减 1，此时 W0 内容是多少呢？ W0 是 16 位寄存器，这里的指令是 16 位操作。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。



结果是-1，但是负数存储的补码的形式存储的。也就是 1 取反，加 1。所以，存储在寄存器中的数值就是 0xffff 。

```
mov     #4,W1
clr     W0
dec     W0,W0
bra     NZ,$-2
dec     W1,W1
bra     NZ,$-8
```

接着来看，这条指令，bra NZ,$-2。bra 呢，是一条条件跳转指令。这里的条件呢，就是 NZ。我们都知道，进行了数学运算之后，结果会影响到对应的标志位。例如，溢出、结果是负数、结果是 0，减法是否有借位等等。

Z 标志位，就是用于判断数学运算的结果是否是 0 。如果 Z = 1，则说明数学运算的结果是 0，否则，说明数学运算的结果非 0 。而这里的 NZ，也就是 Z 标志位不为 1，然后条件跳转。也就是运算结果不为 0，则跳转 。那么，跳转到哪里呢？

```
mov     #4,W1
clr     W0
dec     W0,W0
bra     NZ,$-2
dec     W1,W1
bra     NZ,$-8
```

红圈这个就是要跳转的地址。$表示当前地址。$-2 表示当前地址的上一条地址。也就是说，如果前面的减法运算结果不是 0，那么这里条件判断之后，就会跳回到减法指令。这里两条指令联合起来，就是判断 W0 是否减为 0 了。如果不是 0，那么就不断的判断跳转。如果是 0，则顺序往下执行下一条指令。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
mov     #4, W1
clr     W0
dec     W0, W0
bra     NZ, $-2
dec     W1, W1
bra     NZ, $-8
```

所以，这两条指令是联合起来的。dec 指令，是减法，减法的话，结果是存在 W0 中的，所以就是判断减法的结果是否为 0。

问：是因为是 16 位芯片，每条指令的地址是相差 2 吗？
答：是的，每条指令 24 位，占用两个地址空间。

```
mov     #4, W1
clr     W0
dec     W0, W0
bra     NZ, $-2
dec     W1, W1
bra     NZ, $-8
```

好了，这样的话，我们就清楚了这两条指令的意思。同理呢，我们来看后面的两条指令

```
mov     #4, W1
clr     W0
dec     W0, W0
bra     NZ, $-2
dec     W1, W1
bra     NZ, $-8
```

思路呢，是一样的。这里是判断 W1 是否减为 0. 如果不为 0，则会跳转到 clr w0 指令处。实际上，这里就相当于做了两个 for 循环。一个外循环，一个内循环。外循环的循环次数是 4，内循环的循环次数是 65536 次。这样组合起来呢，就相当于是做了一个延时等待。所以，这段代码的目的呢，就是实现了一个延时。那么总共延时多少时间？这算是给大家留个作业，计算延时多长时间，下节课我们就讲这个。

问：时钟频率多少？
答：时钟频率大家先自己思考，下节课揭晓答案。

那么延时的意义在哪呢？开始上电时，系统电压可能是不稳定的，这时候单片机内部和外部都不太稳定，所以需要延时一会等系统稳定了再工作。

问：可以在主程序中加延时吗？
答：可以，一般需要在时钟和外设配置前加延时。

问：$-8 这个为何要后退 8 个位？
答：因为每条指令是 24 位长度，占 2 个地址空间，低 16 位是偶数地址，高 8 位是奇数地址。所以$-8 要后退 4 条指令。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
mov     #4, W1
clr     W0
dec     W0, W0
bra     NZ, $-2
dec     W1, W1
bra     NZ, $-8
```

要知道这段代码运行多长时间，可以有哪些方法？算指令周期，算是方法 A。用示波器看 IO 输出，算是方法 B。使用定时器也算是另一种方法 C。

这里我们重点讲解第一种方法，使用计算指令周期的方法。倒不是一定要去这样计算，而是因为这计算的背后涉及到较多的单片机知识。

```
28  mov     #4, W1
29  clr     W0
30  dec     W0, W0
31  bra     NZ, $-2
32  dec     W1, W1
33  bra     NZ, $-8
```

先看第一条指令，这条指令呢，是一条普通的传送指令。

MOV	#lit16,Wnd	将 16 位立即数传送至 Wnd	1	1	284

它对应的指令周期是 1 个周期数。

3.3 指令集汇总表

表 3-2： 传送指令

汇编语法		说明	字数	周期数	页码
EXCH	Wns,Wnd	交换 Wns 和 Wnd 的内容	1	1	243
MOV	f {,WREG}(1)	将 f 中内容传送至目标寄存器	1	1	279
MOV	WREG,f	将 WREG 中内容传送至 f	1	1	280
MOV	f,Wnd	将 f 中内容传送至 Wnd	1	1(4)	281
MOV	Wns,f	将 Wns 中内容传送至 f	1	1	282
MOV.B	#lit8,Wnd	将 8 位立即数传送至 Wnd	1	1	283
MOV	#lit16,Wnd	将 16 位立即数传送至 Wnd	1	1	284
MOV	[Ws+Slit10],Wnd	将 [Ws + 有符号 10 位偏移量] 中内容传送至 Wnd	1	1(4)	285
MOV	Wns,[Wd+Slit10]	将 Wns 中内容传送至 [Wd + 有符号 10 位偏移量]	1	1	286
MOV	Wso,Wdo	将 Wso 中内容传送至 Wdo	1	1(4)	287
MOV.D	Ws,Wnd	双字传送 Ws 内容至 Wnd:Wnd + 1	1	2(4)	289
MOV.D	Wns,Wd	双字传送 Wns:Wns + 1 内容至 Wd	1	2	289
MOVPAG	#lit10,DSRPAG(2)	将 10 位立即数传送至 DSRPAG	1	1	291
MOVPAG	#lit9,DSWPAG(2)	将 9 位立即数传送至 DSWPAG	1	1	291
MOVPAG	#lit8,TBLPAG(2)	将 8 位立即数传送至 TBLPAG	1	1	291
MOVPAG	Wn, DSRPAG(2)	将 Wn 中内容传送到 DSRPAG	1	1	292
MOVPAG	Wn, DSWPAG(2)	将 Wn 中内容传送到 DSWPAG	1	1	292
MOVPAG	Wn, TBLPAG(2)	将 Wn 中内容传送到 TBLPAG	1	1	292
SWAP	Wn	Wn = Wn 内容字节或半字节交换	1	1	426
TBLRDH	[Ws],Wd	将高位程序字读入 Wd	1	2(3)	427
TBLRDL	[Ws],Wd	将低位程序字读入 Wd	1	2(3)	429
TBLWTH	Ws,[Wd]	将 Ws 中的内容写入高位程序字	1	2(4)	431
TBLWTL	Ws,[Wd]	将 Ws 中的内容写入低位程序字	1	2(4)	433

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
mov     #4, W1    ;1个指令周期
clr     W0
dec     W0, W0
bra     NZ, $-2
dec     W1, W1
bra     NZ, $-8
```

然后，clr 指令也是一个指令周期。

```
mov     #4, W1    ;1个指令周期
clr     W0        ;1个指令周期
dec     W0, W0
bra     NZ, $-2
dec     W1, W1
bra     NZ, $-8
```

DEC	Ws,Wd	Wd = Ws – 1	1	1(5)	218

这条减法指令，也是一个指令周期。

```
mov     #4, W1    ;1个指令周期
clr     W0        ;1个指令周期
dec     W0, W0    ;1个指令周期
bra     NZ, $-2
dec     W1, W1
bra     NZ, $-8
```

这条条件跳转指令，是有其它说法的。

BRA	NZ,Expr	如果非零则转移	1	1 (2)(1,8)	145

就说，这条指令，如果没有跳转的情况，就是一个指令周期，但是呢，如果有跳转，就需要 4 个指令周期。

REPEAT	#lit14(3)	重复执行下一条指令 (lit14 + 1) 次	1	1	35

注
1：如果未发生转移，则条件转移指令的执行时间为 1 个周期；而如果发生转移，则为 2 个周期。
2：RETURN 指令的执行时间为 3 个周期，但如果有一个异常等待处理，则执行时间为 2 个周期。
3：该指令仅在 dsPIC30F、dsPIC33F 和 dsPIC33E 器件中可用。
4：该指令仅在 dsPIC33E 和 PIC24E 器件中可用。
5：该指令仅在 PIC24F、PIC24H、dsPIC30F 和 dsPIC33F 器件中可用。
6：该指令仅在 dsPIC30F 和 dsPIC33F 器件中可用。
7：该指令仅在 dsPIC33E 器件中可用。
8：在 dsPIC33E 和 PIC24E 器件中，当发生转移时，这些指令需要 2 个额外的周期（总共 4 个周期）。
9：在 dsPIC33E 和 PIC24E 器件中，这些指令需要 3 个额外的周期。

这是注释 1 跟注释 8 对应的指令周期说明。

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

问：dec 操作 W0 属于通用寄存器操作吧，

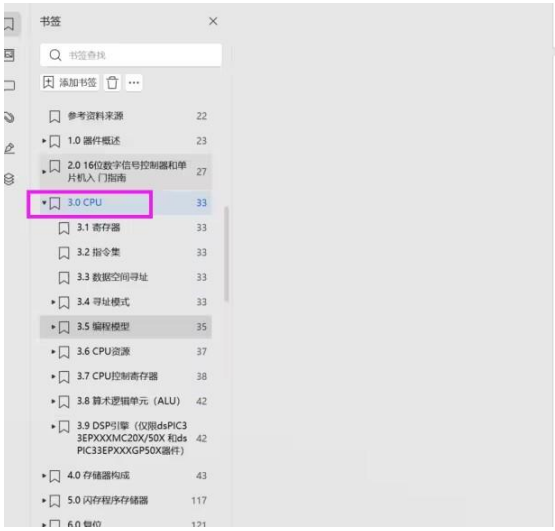
5： 在 dsPIC33E 和 PIC24E 器件中，对非 CPU 特殊功能寄存器的读操作和读 - 修改 - 写操作，都需要 1 个额外周期 —— 与 dsPIC30F、dsPIC33F、PIC24F 和 PIC24H 器件相比。

不是说额外需要一个吗？

答：W0 属于 CPU 寄存器。

问：CPU 特殊功能寄存器是哪些呢？

答：




DS70657G_CN 第 34 页 © 2012-2013 Microchip Technology Inc.

dsPIC33EPXXXGP50X、dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXGP/MC20X

3.5 编程模型

图 3-2 给出了 dsPIC33EPXXXGP50X、dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXGP/MC20X 的编程模型。编程模型中的所有寄存器都是存储器映射的，并且可以通过指令直接操作。表 3-1 列出了对每个寄存器的说明。

除了编程模型中包含的寄存器，dsPIC33EPXXXGP50X、dsPIC33EPXXXMC20X/50X 和 PIC24EPXXXGP/MC20X 器件还包含用于模寻址（仅限 dsPIC33EPXXXMC20X/50X 和 dsPIC33EPXXXGP50X 器件）、位反转寻址（仅限 dsPIC33EPXXXMC20X/50X 和 dsPIC33EPXXXGP50X 器件）和中断的控制寄存器。这些寄存器将在本文档后续章节中进行说明。

与编程模型相关的所有寄存器都是存储器映射的，如表 4-1 中所示。

表 3-1： 编程模型的寄存器说明

寄存器名称	说明
W0 至 W15	工作寄存器阵列
ACCA 和 ACCB	40 位 DSP 累加器
PC	23 位程序计数器
SR	ALU 和 DSP 引擎状态寄存器
SPLIM	堆栈指针限制值寄存器
TBLPAG	表存储器页地址寄存器
DSRPAG	扩展数据空间（EDS）读页寄存器
DSWPAG	扩展数据空间（EDS）写页寄存器
RCOUNT	REPEAT 循环计数寄存器
DCOUNT(1)	DO 循环计数寄存器
DOSTARTH(1,2) 和 DOSTARTL(1,2)	DO 循环起始地址寄存器（高字节和低字节）
DOENDH(1) 和 DOENDL(1)	DO 循环结束地址寄存器（高字节和低字节）
CORCON	包含 DSP 引擎以及 DO 循环控制和陷阱状态位

在 1：…XXXMC20X/50X 和 dsPIC33EPXXXGP50X 器件上可用。
2：DOSTARTH 和 DOSTARTL 寄存器是只读的。

我们用的 MCU 型号是 dsPIC33EP32MC204，所以跳转的时候，指令是 4 个指令周期，没有跳转的时候，是 1 个指令周期。

```
mov     #4,W1    ;1个指令周期
clr     W0       ;1个指令周期
dec     W0,W0    ;1个指令周期
bra     NZ,$-2   ;未跳转是1个指令周期，跳转是4个指令周期
dec     W1,W1    ;1个指令周期
bra     NZ,$-8   ;未跳转是1个指令周期，跳转是4个指令周期
```

这样话，我们把这段代码中，每条指令单独执行可能需要的指令周期数列出来了。接着呢，我们需要把整段代码执行的总的指令周期数给写出来。这样，总的指令周期数知道了，如果再知道单周期指令的时间，那么这段代码的延时时间也就清楚了。

```
mov     #4,W1    ;1个指令周期
clr     W0       ;1个指令周期
dec     W0,W0    ;1个指令周期
bra     NZ,$-2   ;未跳转是1个指令周期，跳转是4个指令周期
dec     W1,W1    ;1个指令周期
bra     NZ,$-8   ;未跳转是1个指令周期，跳转是4个指令周期
```

这条指令，整段代码执行的时候，它会执行多少次啊?

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

```
mov     #4,W1    ;1个指令周期
clr     W0       ;1个指令周期
dec     W0,W0    ;1个指令周期
bra     NZ,$-2   ;未跳转是1个指令周期，跳转是4个指令周期
dec     W1,W1    ;1个指令周期
bra     NZ,$-8   ;未跳转是1个指令周期，跳转是4个指令周期
```

W1 是 4 的时候执行 1 次，接着减到 0 之前，会再执行 3 次，所以，总的 clr W0 指令会执行 4 次。

```
mov     #4,W1    ;1个指令周期
clr     W0       ;1个指令周期 * 4
dec     W0,W0    ;1个指令周期
bra     NZ,$-2   ;未跳转是1个指令周期，跳转是4个指令周期
dec     W1,W1    ;1个指令周期
bra     NZ,$-8   ;未跳转是1个指令周期，跳转是4个指令周期
```

所以，这条指令在这段代码中的指令周期数就是 4 个。

```
mov     #4,W1    ;1个指令周期
clr     W0       ;1个指令周期 * 4
dec     W0,W0    ;1个指令周期
bra     NZ,$-2   ;未跳转是1个指令周期，跳转是4个指令周期
dec     W1,W1    ;1个指令周期
bra     NZ,$-8   ;未跳转是1个指令周期，跳转是4个指令周期
```

然后，再来看这条指令，单独考虑内循环，这条指令会执行多少次呢?

65536 次内循环，因为 W0 从 0 开始执行。然后，再考虑外循环跟它的联系。外循环考虑之后，这条指令的执行次数就是 65536 * 4 次。

```
mov     #4,W1    ;1个指令周期
clr     W0       ;1个指令周期 * 4
dec     W0,W0    ;1个指令周期 * 65536 * 4
bra     NZ,$-2   ;未跳转是1个指令周期，跳转是4个指令周期
dec     W1,W1    ;1个指令周期
bra     NZ,$-8   ;未跳转是1个指令周期，跳转是4个指令周期
```

接着来看这条指令呢，在一个内循环中，它有多少次是 4 个指令周期，有多少次是 1 个指令周期? 是不是应该是 65535 次 4 个周期，加上 1 个指令周期啊? W0 从 0 开始减到 1，都会发生跳转。而 1 减 0 的时候，结果已经是 0 了，所以不会跳转。因此，应该是 65535*4 + 1，

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

这是 bra 指令内循环的执行周期数。 再考虑外循环的话，就是(65535*4 + 1) * 4，对吧？

```
mov     #4,W1     ;1个指令周期
clr     W0        ;1个指令周期 * 4
dec     W0,W0     ;1个指令周期 * 65536 * 4
bra     NZ,$-2    ;未跳转是1个指令周期，跳转是4个指令周期   (65535 * 4 + 1) * 4
dec     W1,W1     ;1个指令周期
bra     NZ,$-8    ;未跳转是1个指令周期，跳转是4个指令周期
```

那么，这条指令呢？是不是这段代码，执行的周期数就是 4 啊？

```
mov     #4,W1     ;1个指令周期
clr     W0        ;1个指令周期 * 4
dec     W0,W0     ;1个指令周期 * 65536 * 4
bra     NZ,$-2    ;未跳转是1个指令周期，跳转是4个指令周期   (65535 * 4 + 1) * 4
dec     W1,W1     ;1个指令周期 * 4
bra     NZ,$-8    ;未跳转是1个指令周期，跳转是4个指令周期
```

那么，最后一条指令，显然就是 3 次跳转加 1 次不跳转，所以就是 3*4 + 1。

```
mov     #4,W1     ;1个指令周期
clr     W0        ;1个指令周期 * 4
dec     W0,W0     ;1个指令周期 * 65536 * 4
bra     NZ,$-2    ;未跳转是1个指令周期，跳转是4个指令周期   (65535 * 4 + 1) * 4
dec     W1,W1     ;1个指令周期 * 4
bra     NZ,$-8    ;未跳转是1个指令周期，跳转是4个指令周期    3*4 + 1
```

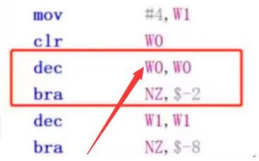
这样的话，这段代码执行完，需要的总的周期数就全部求解出来了。1+4+65536*4+（65535*4+1）*4+4+（3*4+1）=5+262144+（262140+1）*4+4+13=1310730（指令周期）。

那么，具体时间是多少呢？这里，还需要知道一个单周期指令的时间。

举个例子，假设单周期指令的时间是 1/60MHz=16.667ns，则这段延时代码对应的时间就是 16.667ns* 1310730 = 21845936.91ns = 21.84593691ms。

关于分析实际代码的单指令的执行时间，是我们下节课讲解的内容，大家课后可以先查资料思考。

问：这里面也没其他语句，怎么知道是判断 W0 的呢？判断 W0 不为 0？

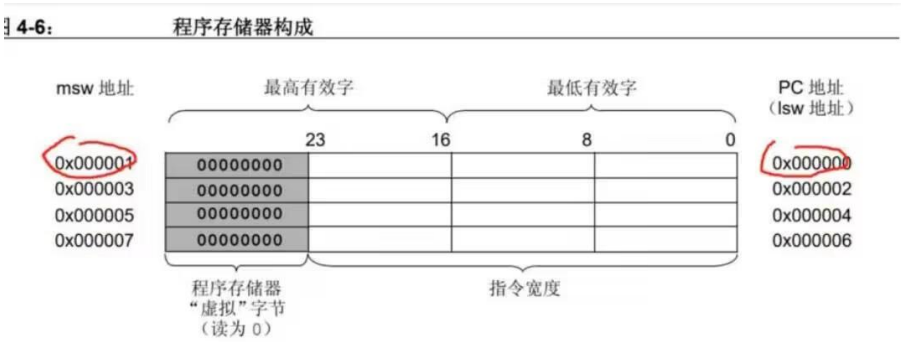


本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。

答：数学运算指令的结果影响 Z 标志位，是判断运算结果，不同的指令只看运算结果，不一定是 W0。

一会等系统稳定了再工作。(提问：因为每条指令是 24 位长度，占 2 个地址空间，低 16 位是偶数地址，高 8 位是奇数地址。所以$-8 要后退 4 条指令)。

问：
老师这句话什么意思？



答：

今天内容就到这里，谢谢大家，晚安！

本文件为保密资料，仅供学员本人学习，严禁传播复制以及作为商业用途，违反必究，后果自负。